RETAILMART V3 · SQL

Constraints, Keys & DML

— Siraj Sir

📱 WhatsApp Announcement

CONSTRAINTS, KEYS & DML
Make Your Tables Bulletproof!

A database without constraints is just an expensive Excel sheet. Today we learn how to make our tables reject bad data automatically.

Today's Focus:

- NOT NULL, DEFAULT, UNIQUE, CHECK — the data shields
- PRIMARY KEY & FOREIGN KEY — identity and relationships
- CASCADE vs RESTRICT — what happens when parents are deleted?
- INSERT, UPDATE, DELETE, TRUNCATE — real data operations

ALL practice happens in YOUR database (accio_26), NOT RetailMart.
Course Portal: https://starlordali4444.github.io/PostgreSql/

Constraints = interview gold. See you ON TIME!
— Siraj Sir #Day4 #Constraints #DML

RECAP

YOUR SQL JOURNEY SO FAR

SQL Intro & PostgreSQL Architecture

- SQL is declarative — you say WHAT, engine decides HOW
- 5 SQL categories: DDL, DML, DQL, DCL, TCL

Installation & RetailMart Onboarding

- PostgreSQL 18, pgAdmin 4, VS Code installed
- RetailMart V3 loaded — 16 schemas, 55 tables

DDL & Data Types

- CREATE, ALTER, DROP — structure commands
- INT, NUMERIC, VARCHAR, TIMESTAMP, BOOLEAN — type choices

Today: Making tables BULLETPROOF with Constraints + filling them with DML

TODAY'S AGENDA (2 Hours)

Section	Time	Topics
Part 1: Basic Constraints (40 mins)	40 mins	NOT NULL, DEFAULT, UNIQUE, CHECK
Part 2: Primary Keys & Foreign Keys (30 mins)	30 mins	PRIMARY KEY — single, composite, UUID, FOREIGN KEY — referential integrity
Part 3: Cascading Actions (15 mins)	15 mins	ON DELETE CASCADE / RESTRICT / SET NULL
Part 4: DML — Data Manipulation (35 mins)	35 mins	INSERT INTO, UPDATE, DELETE, TRUNCATE

STORY

The School Exam Hall Without Rules

A school organizes exams with NO rules. Chaos follows:

- Two students sit in the same seat — fight breaks out (No UNIQUE)
- A student's name on the admit card is blank (No NOT NULL)
- Roll Number: -47 is accepted (No CHECK)
- Hall 99 doesn't exist but the admit card says it (No FOREIGN KEY)

The vice principal fixes it: Every seat gets a unique number (PK). Every student must have a name (NOT NULL). Roll numbers must be positive (CHECK). Hall numbers must match the actual list (FK).

Next exam — zero chaos. Those rules? In SQL, they're called CONSTRAINTS.

STORY

STORY to SQL CONNECTION

- Same seat = Duplicate rows -> UNIQUE prevents this
- Blank name = NULL in required column -> NOT NULL rejects it
- Negative roll = Invalid value -> CHECK blocks it
- Hall 99 = Orphaned reference -> FOREIGN KEY ensures parent exists
- Seat number = PRIMARY KEY (UNIQUE + NOT NULL combined)

REAL WORLD

REAL-WORLD CONNECTION

Flipkart, Amazon India, Myntra — all learned this the hard way. Application code can be bypassed, but database constraints CANNOT. A single CHECK on price > 0 saves crores.

STORY

The Zomato Delivery Disaster

You order Biryani on Zomato at 10 PM. Payment deducted. Delivery partner picks up the food. He opens his app for the address… BLANK. The address column has NULL.

Food gets cold. Customer angry. All because one column did not have NOT NULL.
NOT NULL is like a bouncer at a club. No ID? You're NOT getting in. Period.

DEFINITION

NOT NULL

Technical: NOT NULL prohibits NULL values in a column. Any INSERT or UPDATE setting the column to NULL is rejected with an error.

Layman's: Like a government form where the Aadhaar field has a red asterisk (*). You cannot submit without filling it in.

SYNTAX: NOT NULL

```
-- During CREATE TABLE
CREATE TABLE schema.table (
    column_name DATA_TYPE NOT NULL
);

-- Add to existing table
ALTER TABLE schema.table
ALTER COLUMN col SET NOT NULL;

-- Remove constraint
ALTER TABLE schema.table
ALTER COLUMN col DROP NOT NULL;
```

Easy: Your First NOT NULL Table

EASY

SCENARIO: You are building a user registration system for a college portal. Every student MUST have a username and email — blank entries should be impossible.

TASK: Create a day_4.users table with user_id (SERIAL PK), username (VARCHAR NOT NULL), and email (VARCHAR NOT NULL). Insert one valid row, then try inserting a row without a username.

HINT: When you skip a NOT NULL column in INSERT, PostgreSQL throws 'null value in column ... violates not-null constraint'.

Answer

✅ ANSWER

```
CREATE SCHEMA IF NOT EXISTS day_4;

CREATE TABLE day_4.users (
    user_id SERIAL PRIMARY KEY,
    username VARCHAR(50) NOT NULL,
    email VARCHAR(150) NOT NULL
);

-- SUCCESS: Both required fields provided
INSERT INTO day_4.users (username, email)
VALUES ('rahul_sharma', 'rahul@gmail.com');

-- FAILS: username is missing!
INSERT INTO day_4.users (email)
VALUES ('someone@gmail.com');
-- ERROR: null value in column "username"
-- violates not-null constraint
```

Medium: Stores With Mandatory Fields

MEDIUM

SCENARIO: RetailMart is opening new stores across India. The operations team needs every store to have a name, city, and pincode in the database — but phone number is optional since some kiosks don't have landlines.

TASK: Create day_4.stores with store_id (SERIAL PK), store_name (NOT NULL), city (NOT NULL), pincode (NOT NULL), and phone (nullable). Insert a valid store, then try inserting one without a city.

HINT: Only columns WITHOUT the NOT NULL keyword accept NULL. Phone is intentionally nullable here.

Answer

✅ ANSWER

```
CREATE TABLE day_4.stores (
    store_id SERIAL PRIMARY KEY,
    store_name VARCHAR(100) NOT NULL,
    city VARCHAR(50) NOT NULL,
    pincode VARCHAR(6) NOT NULL,
    phone VARCHAR(15)  -- optional
);

-- SUCCESS
INSERT INTO day_4.stores (store_name, city, pincode)
VALUES ('RetailMart Andheri', 'Mumbai', '400058');

-- FAILS: city is NULL
INSERT INTO day_4.stores (store_name, pincode)
VALUES ('RetailMart Unknown', '400001');
-- ERROR: null value in column "city"
```

Hard: Adding NOT NULL to an Existing Table

HARD

SCENARIO: Your warehouse table was created WITHOUT NOT NULL on city. Some rows already have NULL cities. Your manager says: 'Fix this — every warehouse must have a city from now on.'

TASK: Create day_4.warehouses without NOT NULL on city. Insert two rows (one with city, one without). Try adding NOT NULL — it will fail. Fix the existing NULLs, then add the constraint.

HINT: You cannot add NOT NULL if existing rows already have NULL. Use UPDATE … SET city = 'Unknown' WHERE city IS NULL first.

Answer (1/2)

✅ ANSWER

```
CREATE TABLE day_4.warehouses (
    warehouse_id SERIAL PRIMARY KEY,
    warehouse_name VARCHAR(100),
    city VARCHAR(50)
);

INSERT INTO day_4.warehouses (warehouse_name, city)
VALUES ('Warehouse Alpha', 'Delhi');
INSERT INTO day_4.warehouses (warehouse_name)
VALUES ('Warehouse Beta');  -- city is NULL

-- This FAILS: existing NULLs block it
ALTER TABLE day_4.warehouses
ALTER COLUMN city SET NOT NULL;
-- ERROR: column contains null values

-- Fix NULLs first
UPDATE day_4.warehouses
SET city = 'Unknown' WHERE city IS NULL;

-- Now it succeeds
ALTER TABLE day_4.warehouses
```

Answer (2/2)

✅ ANSWER

```
ALTER COLUMN city SET NOT NULL;
```

Crazy: Multiple NOT NULL on a Product Listing

CRAZY

SCENARIO: You are building Meesho's product listing table. Every listing MUST have a product name, seller name, price, stock quantity, and category. Only the description is optional.

TASK: Create day_4.product_listings with 5 NOT NULL columns and 1 optional column. Try inserting a row with only the product_name — observe how many constraint errors you get.

HINT: PostgreSQL stops at the FIRST constraint violation. Fix one, and the next NOT NULL column will fail until all are provided.

Answer

✅ ANSWER

```
CREATE TABLE day_4.product_listings (
    listing_id SERIAL PRIMARY KEY,
    product_name VARCHAR(200) NOT NULL,
    seller_name VARCHAR(100) NOT NULL,
    price NUMERIC(10,2) NOT NULL,
    stock_qty INT NOT NULL,
    category VARCHAR(50) NOT NULL,
    description TEXT  -- optional
);

-- FAILS: missing seller_name
INSERT INTO day_4.product_listings (product_name)
VALUES ('Wireless Mouse');

-- SUCCESS: all NOT NULL columns provided
INSERT INTO day_4.product_listings
  (product_name, seller_name, price, stock_qty, category)
VALUES ('Wireless Mouse', 'TechZone India',
        599.00, 150, 'Electronics');
```

KEY TAKEAWAYS

NOT NULL

NOT NULL ensures no missing data in critical columns. Ask: 'Can this row exist meaningfully without this column?' If NO, add NOT NULL.

COMMON MISTAKE

Forgetting NOT NULL on Critical Columns

The Mistake: Defining email VARCHAR(150), phone VARCHAR(15) without NOT NULL — then wondering why users have no contact info.

The Fix: A user without an email is not a real user. Add NOT NULL to every essential column.

STORY

The Dunzo Midnight Mystery

Dunzo had a bug: every midnight, hundreds of orders appeared with order_status = NULL. The app crashed during checkout and the API inserted rows without a status.

The fix: DEFAULT 'pending' on the status column. Now even crashed checkouts get a proper status automatically. DEFAULT is like Google Form auto-fill — leave a field blank, it fills in a sensible value.

DEFINITION

DEFAULT

Technical: DEFAULT specifies a value PostgreSQL automatically inserts when an INSERT does not provide one. Can be a literal, expression, or function like NOW().

Layman's: Like a job application form where 'Preferred Location' auto-fills to 'Any Location' if left blank.

SYNTAX: DEFAULT

```
column_name DATA_TYPE DEFAULT value
-- Examples:
status VARCHAR(20) DEFAULT 'pending'
is_active BOOLEAN DEFAULT TRUE
created_at TIMESTAMPTZ DEFAULT NOW()
session_id UUID DEFAULT gen_random_uuid()

-- Add/remove default on existing table
ALTER TABLE t ALTER COLUMN c SET DEFAULT 'value';
ALTER TABLE t ALTER COLUMN c DROP DEFAULT;
```

Easy: Orders With Auto-Fill Status

EASY

SCENARIO: A food delivery app needs every order to have a status. But the mobile app sometimes crashes before setting it. You want the database to auto-fill 'pending' and timestamp the creation.

TASK: Create day_4.orders with order_status DEFAULT 'pending' and created_at DEFAULT NOW(). Insert a row providing only customer_name and verify the defaults filled in.

HINT: DEFAULT kicks in only when you DON'T provide the column in your INSERT. If you explicitly pass NULL, DEFAULT does NOT apply.

Answer

✅ ANSWER

```
CREATE TABLE day_4.orders (
    order_id SERIAL PRIMARY KEY,
    customer_name VARCHAR(100) NOT NULL,
    order_status VARCHAR(20) DEFAULT 'pending',
    created_at TIMESTAMPTZ DEFAULT NOW()
);

-- Insert without status or timestamp
INSERT INTO day_4.orders (customer_name)
VALUES ('Priya Mehta');

SELECT * FROM day_4.orders;
-- order_status = 'pending' (auto-filled)
-- created_at = current timestamp (auto-filled)
```

Medium: NOT NULL + DEFAULT — The Power Combo

MEDIUM

SCENARIO: You are building a products table. Every product must have is_active, stock_qty, and created_at values — but developers often forget to set them. You want mandatory values that auto-fill.

TASK: Create day_4.products with NOT NULL DEFAULT on is_active (TRUE), stock_qty (0), and created_at (NOW()). Insert a row with only name and price.

HINT: NOT NULL + DEFAULT = column MUST have a value, but the developer doesn't have to provide it. Best of both worlds.

Answer

✅ ANSWER

```
CREATE TABLE day_4.products (
    product_id SERIAL PRIMARY KEY,
    product_name VARCHAR(200) NOT NULL,
    price NUMERIC(10,2) NOT NULL,
    is_active BOOLEAN NOT NULL DEFAULT TRUE,
    stock_qty INT NOT NULL DEFAULT 0,
    created_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

INSERT INTO day_4.products (product_name, price)
VALUES ('Basmati Rice 5kg', 450.00);

-- is_active=TRUE, stock_qty=0, created_at=now
-- All auto-filled by DEFAULT!
```

Hard: ALTER TABLE to Add DEFAULT Later

HARD

SCENARIO: The orders table already exists in production without a default on order_status. New orders are coming in with NULL status. Your task is to add the default without breaking existing data.

TASK: Use ALTER TABLE to add DEFAULT 'pending' to order_status. Then verify that NEW inserts get the default but existing NULL rows are NOT changed.

HINT: SET DEFAULT only affects FUTURE inserts. Existing NULL rows must be fixed with a separate UPDATE command.

Answer

✅ ANSWER

```
ALTER TABLE day_4.orders
ALTER COLUMN order_status SET DEFAULT 'pending';

-- New inserts auto-fill 'pending'
-- But existing NULL rows are NOT fixed!
-- Fix them manually:
UPDATE day_4.orders
SET order_status = 'pending'
WHERE order_status IS NULL;

-- To remove a default later:
ALTER TABLE day_4.orders
ALTER COLUMN order_status DROP DEFAULT;
```

Crazy: Function-Based DEFAULTs for Audit Tables

CRAZY

SCENARIO: Your company needs an audit_log table that auto-fills: timestamp (NOW), date (CURRENT_DATE), a unique session ID (UUID), resolved flag (FALSE), and priority (1) — the developer only provides the event type.

TASK: Create day_4.audit_log with 5 auto-filling columns using NOW(), CURRENT_DATE, gen_random_uuid(), FALSE, and 1 as defaults. Insert a row with just event_type.

HINT: PostgreSQL DEFAULT can use any valid expression or function — not just static values.

Answer

✅ ANSWER

```
CREATE TABLE day_4.audit_log (
    log_id SERIAL PRIMARY KEY,
    event_type VARCHAR(50) NOT NULL,
    log_time TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    log_date DATE NOT NULL DEFAULT CURRENT_DATE,
    session_id UUID NOT NULL DEFAULT gen_random_uuid(),
    is_resolved BOOLEAN NOT NULL DEFAULT FALSE,
    priority INT NOT NULL DEFAULT 1
);

INSERT INTO day_4.audit_log (event_type)
VALUES ('LOGIN_ATTEMPT');
-- All 5 columns auto-filled!
```

KEY TAKEAWAYS

DEFAULT

DEFAULT provides sensible fallbacks. Pair NOT NULL + DEFAULT for columns that must always have a value. Common defaults: NOW() for timestamps, FALSE for booleans, 0 for counters, 'pending' for status.

PRO TIP

NOT NULL + DEFAULT = Self-Documenting Tables

Even if a junior dev forgets to set a value, the database fills in the correct default. Your CREATE TABLE becomes the single source of truth.

CHALLENGE

MINI CHALLENGE: NOT NULL + DEFAULT

1.: Create day_4.blog_posts with: post_id SERIAL PK, title VARCHAR NOT NULL, status VARCHAR DEFAULT 'draft', created_at TIMESTAMPTZ NOT NULL DEFAULT NOW().

2.: Insert a row with only the title. Verify defaults filled in.

3.: Try inserting without a title — observe the NOT NULL error.

STORY

The PhonePe OTP Chaos

PhonePe has 50 crore users. Imagine if TWO users register with the same phone number. User B exploits a race condition and also gets 9876543210. An OTP for a Rs. 50,000 transaction goes to BOTH. User B approves User A's transaction. Money gone.

UNIQUE constraint makes it physically impossible for two rows to share a phone number. Like Indian Railways seat reservation — once booked, no one else can get that seat.

DEFINITION

UNIQUE

Technical: UNIQUE ensures all values in a column (or column combination) are distinct. PostgreSQL creates a B-tree index internally. Duplicate INSERT/UPDATE is rejected.

Layman's: Like your Aadhaar number — no two people can share it. UNIQUE does this for any column: emails, phones, PAN numbers.

SYNTAX: UNIQUE

```
-- Column level
column_name DATA_TYPE UNIQUE

-- Multi-column (table level)
UNIQUE (col_a, col_b)

-- Add to existing table
ALTER TABLE t ADD CONSTRAINT name UNIQUE (col);
```

Easy: Newsletter — Each Email Must Be Unique

EASY

SCENARIO: You are building a newsletter subscription system. The same email should never appear twice — duplicate subscriptions waste money on double emails.

TASK: Create day_4.subscribers with email UNIQUE NOT NULL. Insert one subscriber, then try inserting the same email again.

HINT: The error message includes the constraint name like 'subscribers_email_key' — this tells you exactly which UNIQUE constraint was violated.

Answer

✅ ANSWER

```
CREATE TABLE day_4.subscribers (
    subscriber_id SERIAL PRIMARY KEY,
    email VARCHAR(150) UNIQUE NOT NULL
);

INSERT INTO day_4.subscribers (email)
VALUES ('priya@gmail.com');  -- Works

INSERT INTO day_4.subscribers (email)
VALUES ('priya@gmail.com');  -- BLOCKED
-- ERROR: duplicate key value violates
-- unique constraint "subscribers_email_key"
```

Medium: Multiple UNIQUE Columns on One Table

MEDIUM

SCENARIO: An employee table at Infosys needs three uniqueness rules: no duplicate emails, no duplicate PAN numbers, and no duplicate phone numbers. Each is independently unique.

TASK: Create day_4.employees with emp_email, pan_number (both UNIQUE NOT NULL), and phone (UNIQUE but nullable). Insert one employee, then try inserting another with the same PAN.

HINT: A table can have MULTIPLE UNIQUE constraints. A duplicate in ANY of them causes rejection independently.

Answer

✅ ANSWER

```
CREATE TABLE day_4.employees (
    emp_id SERIAL PRIMARY KEY,
    emp_email VARCHAR(150) UNIQUE NOT NULL,
    pan_number VARCHAR(10) UNIQUE NOT NULL,
    phone VARCHAR(15) UNIQUE,
    emp_name VARCHAR(100) NOT NULL
);

INSERT INTO day_4.employees
  (emp_email, pan_number, phone, emp_name)
VALUES ('amit@infosys.com', 'ABCDE1234F',
        '9876543210', 'Amit Kumar');

-- FAILS: same PAN number
INSERT INTO day_4.employees
  (emp_email, pan_number, phone, emp_name)
VALUES ('raj@tcs.com', 'ABCDE1234F',
        '9123456789', 'Raj Singh');
```

Hard: Multi-Column UNIQUE — One Coupon Per User

HARD

SCENARIO: Flipkart runs a DIWALI50 coupon. Each user can use it only ONCE, but different users can all use it. Neither user_id alone nor coupon_code alone is unique — but the COMBINATION must be.

TASK: Create day_4.coupon_usage with UNIQUE (user_id, coupon_code). Show that user 1 + DIWALI50 works, user 2 + DIWALI50 works, but user 1 + DIWALI50 again is blocked.

HINT: Multi-column UNIQUE means the combination must be unique. (1, DIWALI50) and (2, DIWALI50) are different combinations.

Answer

✅ ANSWER

```
CREATE TABLE day_4.coupon_usage (
    usage_id SERIAL PRIMARY KEY,
    user_id INT NOT NULL,
    coupon_code VARCHAR(20) NOT NULL,
    used_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    UNIQUE (user_id, coupon_code)
);

INSERT INTO day_4.coupon_usage (user_id, coupon_code)
VALUES (1, 'DIWALI50');  -- Works
INSERT INTO day_4.coupon_usage (user_id, coupon_code)
VALUES (2, 'DIWALI50');  -- Works (different user)
INSERT INTO day_4.coupon_usage (user_id, coupon_code)
VALUES (1, 'DIWALI50');  -- BLOCKED! Same combo
```

Crazy: UNIQUE and NULLs — PostgreSQL's Gotcha

CRAZY

SCENARIO: A profiles table has an optional nickname column with UNIQUE. Two users leave nickname blank (NULL). Should PostgreSQL block the second NULL as a 'duplicate'?

TASK: Create day_4.profiles with nickname UNIQUE (nullable). Insert two rows with NULL nickname — both succeed. Then insert two rows with the same actual nickname — second is blocked.

HINT: In PostgreSQL, NULL != NULL. Two unknowns are NOT considered duplicates. This is a classic interview question!

Answer

✅ ANSWER

```
CREATE TABLE day_4.profiles (
    profile_id SERIAL PRIMARY KEY,
    username VARCHAR(50) UNIQUE NOT NULL,
    nickname VARCHAR(50) UNIQUE  -- allows NULL
);

-- Both succeed! NULL != NULL
INSERT INTO day_4.profiles (username)
VALUES ('user_alpha');  -- nickname=NULL
INSERT INTO day_4.profiles (username)
VALUES ('user_beta');   -- nickname=NULL

-- But actual values must be unique:
INSERT INTO day_4.profiles (username, nickname)
VALUES ('user_gamma', 'TheGamer');  -- Works
INSERT INTO day_4.profiles (username, nickname)
VALUES ('user_delta', 'TheGamer');  -- BLOCKED
```

KEY TAKEAWAYS

UNIQUE

UNIQUE = no duplicate values. Use for emails, phones, PAN. Multiple NULLs allowed in PostgreSQL. A table can have many UNIQUE constraints but only one PRIMARY KEY.

INTERVIEW

UNIQUE vs PRIMARY KEY

Q: 'Difference between UNIQUE and PRIMARY KEY?'

- Multiple UNIQUE per table, only ONE PRIMARY KEY
- UNIQUE allows NULL; PRIMARY KEY is strictly NOT NULL
- PRIMARY KEY = definitive row identity; UNIQUE = business-level uniqueness

STORY

The Swiggy Rating Nightmare

Swiggy lets users rate 1-5 stars. A buggy API sends rating = 999. Without CHECK, the database accepts it. Average rating shows 200 stars. Dashboard breaks. ML models produce garbage.

With CHECK (rating BETWEEN 1 AND 5), the database rejects 999 instantly. CHECK is like a height requirement on a roller coaster — too short or too tall, you're rejected.

DEFINITION

CHECK

Technical: CHECK evaluates a Boolean expression for every INSERT/UPDATE. If FALSE, the operation is rejected. Can reference one or more columns.

Layman's: A security guard with a checklist. 'Age above 18? Salary positive? Status valid?' If any check fails, data is turned away.

SYNTAX: CHECK

```
-- Column-level
price NUMERIC(10,2) CHECK (price >= 0)

-- Table-level (multi-column)
CONSTRAINT name CHECK (end_date >= start_date)

-- Common patterns
CHECK (rating BETWEEN 1 AND 5)
CHECK (status IN ('a', 'b', 'c'))
CHECK (discount BETWEEN 1 AND 100)
```

Easy: No Negative Prices in Inventory

EASY

SCENARIO: A retail inventory system must never accept negative prices or quantities. A developer accidentally sends price = -10 from a buggy script — the database should block it.

TASK: Create day_4.inventory with price CHECK >= 0 and quantity CHECK >= 0. Insert valid data, then try a negative price.

HINT: The error message says 'violates check constraint' with the auto-generated constraint name like 'inventory_price_check'.

Answer

✅ ANSWER

```
CREATE TABLE day_4.inventory (
    item_id SERIAL PRIMARY KEY,
    item_name VARCHAR(100) NOT NULL,
    price NUMERIC(10,2) CHECK (price >= 0),
    quantity INT CHECK (quantity >= 0)
);

-- SUCCESS
INSERT INTO day_4.inventory (item_name, price, quantity)
VALUES ('Notebook', 45.00, 200);

-- FAILS: negative price
INSERT INTO day_4.inventory (item_name, price, quantity)
VALUES ('Pen', -10.00, 100);
-- ERROR: violates check constraint
```

Medium: Restricting Values to a Set (Dropdown Effect)

MEDIUM

SCENARIO: An order tracking system should only accept statuses: placed, shipped, delivered, cancelled. A rating field must be 1-5. Any other value is a bug or a hack.

TASK: Create day_4.order_tracking with status CHECK IN (...) and rating CHECK BETWEEN 1 AND 5. Try inserting 'dispatched' as status and 7 as rating.

HINT: CHECK with IN (...) acts like a dropdown menu. CHECK with BETWEEN enforces a numeric range.

Answer

✅ ANSWER

```
CREATE TABLE day_4.order_tracking (
    tracking_id SERIAL PRIMARY KEY,
    order_id INT NOT NULL,
    status VARCHAR(20) NOT NULL CHECK (
        status IN ('placed','shipped','delivered','cancelled')
    ),
    rating INT CHECK (rating BETWEEN 1 AND 5)
);

-- SUCCESS
INSERT INTO day_4.order_tracking (order_id, status, rating)
VALUES (101, 'shipped', 4);

-- FAILS: 'dispatched' not in allowed list
INSERT INTO day_4.order_tracking (order_id, status)
VALUES (102, 'dispatched');
```

Hard: Multi-Column CHECK — End Date After Start Date

HARD

SCENARIO: A promotions table needs: discount between 1-100%, and end_date must be AFTER start_date. A promo that ends before it starts makes no sense.

TASK: Create day_4.promotions with CHECK on discount_pct and a table-level CONSTRAINT for date validation. Try inserting a promo where end_date < start_date.

HINT: Multi-column CHECKs must be table-level (after all columns) with the CONSTRAINT keyword.

Answer

✅ ANSWER

```
CREATE TABLE day_4.promotions (
    promo_id SERIAL PRIMARY KEY,
    promo_code VARCHAR(20) NOT NULL,
    discount_pct INT NOT NULL
        CHECK (discount_pct BETWEEN 1 AND 100),
    start_date DATE NOT NULL,
    end_date DATE NOT NULL,
    CONSTRAINT valid_dates
        CHECK (end_date >= start_date)
);

-- SUCCESS
INSERT INTO day_4.promotions
  (promo_code, discount_pct, start_date, end_date)
VALUES ('REPUBLIC25', 25, '2025-01-20', '2025-01-31');

-- FAILS: end before start
INSERT INTO day_4.promotions
  (promo_code, discount_pct, start_date, end_date)
VALUES ('BAD', 50, '2025-01-10', '2025-01-01');
```

Crazy: Complex Multi-Constraint Payroll Table

CRAZY

SCENARIO: A payroll system needs: salary >= 15000 (minimum wage), bonus >= 0, emp_type must be one of 3 values, period_end > period_start, AND bonus cannot exceed salary. Five separate business rules.

TASK: Create day_4.payroll with 3 column-level CHECKs and 2 table-level CONSTRAINTs. Try inserting a salary of 5000.

HINT: Each CHECK is independent. ALL must pass. Column-level for single-column rules, table-level for cross-column rules.

Answer

✅ ANSWER

```
CREATE TABLE day_4.payroll (
    payroll_id SERIAL PRIMARY KEY,
    emp_name VARCHAR(100) NOT NULL,
    base_salary NUMERIC(12,2) NOT NULL
        CHECK (base_salary >= 15000),
    bonus NUMERIC(10,2) NOT NULL DEFAULT 0
        CHECK (bonus >= 0),
    emp_type VARCHAR(20) NOT NULL
        CHECK (emp_type IN ('full_time','part_time','contract')),
    period_start DATE NOT NULL,
    period_end DATE NOT NULL,
    CONSTRAINT valid_period CHECK (period_end > period_start),
    CONSTRAINT bonus_cap CHECK (bonus <= base_salary)
);

-- FAILS: salary 5000 < 15000 minimum
INSERT INTO day_4.payroll
  (emp_name, base_salary, emp_type,
   period_start, period_end)
VALUES ('Intern', 5000, 'part_time',
        '2025-01-01', '2025-01-31');
```

KEY TAKEAWAYS

CHECK

CHECK = custom business rules at the DB level. Ranges (BETWEEN), sets (IN), cross-column logic (end > start). Never rely only on app code for validation.

COMMON MISTAKE

CHECK with NULL

The Mistake: Assuming CHECK (rating BETWEEN 1 AND 5) blocks NULL. It does NOT — NULL makes the expression UNKNOWN, and CHECK only rejects FALSE.

The Fix: Combine NOT NULL CHECK (rating BETWEEN 1 AND 5). NOT NULL blocks NULLs; CHECK blocks invalid values.

STORY

The IRCTC PNR Chaos

Indian Railways handles 2.5 crore passengers DAILY. Without unique PNR numbers, you call support: 'Check my ticket.' They ask your name. 'Raj Kumar.' There are 47,000 Raj Kumars today. Which one?

PNR = PRIMARY KEY. It pinpoints ONE booking out of millions instantly. Like your Aadhaar — unique, never blank, never changes.

DEFINITION

PRIMARY KEY

Technical: PRIMARY KEY = UNIQUE + NOT NULL. Uniquely identifies each row. Only ONE per table. Can be single-column or composite. PostgreSQL auto-creates a B-tree index for fast lookups.

Layman's: The Aadhaar of your table. Every row gets one, no sharing, never blank. When another table needs to point to this row, it points to the PK.

SYNTAX: PRIMARY KEY

```
-- Single column (most common)
id SERIAL PRIMARY KEY

-- Composite (multiple columns)
PRIMARY KEY (col_a, col_b)

-- UUID for distributed systems
id UUID PRIMARY KEY DEFAULT gen_random_uuid()
```

Easy: SERIAL Primary Key — Auto-Increment

EASY

SCENARIO: A customer registration system needs auto-generated unique IDs. The developer should never have to specify the ID manually.

TASK: Create day_4.customers with customer_id SERIAL PRIMARY KEY. Insert two customers without specifying IDs. Verify they got 1 and 2. Try manually inserting id=1 again.

HINT: SERIAL auto-generates 1, 2, 3… You never specify it. Trying to insert a duplicate PK gives 'duplicate key value violates unique constraint'.

Answer

✅ ANSWER

```
CREATE TABLE day_4.customers (
    customer_id SERIAL PRIMARY KEY,
    full_name VARCHAR(100) NOT NULL,
    email VARCHAR(150) UNIQUE NOT NULL
);

INSERT INTO day_4.customers (full_name, email)
VALUES ('Ananya Sharma', 'ananya@gmail.com');
INSERT INTO day_4.customers (full_name, email)
VALUES ('Vikram Patel', 'vikram@gmail.com');
-- IDs auto-assigned: 1, 2

-- FAILS: duplicate PK
INSERT INTO day_4.customers
  (customer_id, full_name, email)
VALUES (1, 'Duplicate', 'dup@gmail.com');
```

Medium: Natural Key vs Surrogate Key

MEDIUM

SCENARIO: A debate at your company — should we use email as the PRIMARY KEY, or an auto-generated integer? What happens if a user changes their email?

TASK: Create day_4.bad_customers with email as PK, and day_4.good_customers with SERIAL PK + email as UNIQUE. Write SQL comments explaining why surrogate is better.

HINT: If email is PK and a user changes email, every other table referencing it must also update — cascading chaos. Surrogate keys never change.

Answer

✅ ANSWER

```
-- BAD: Email as PK (what if user changes email?)
CREATE TABLE day_4.bad_customers (
    email VARCHAR(150) PRIMARY KEY,
    name VARCHAR(100) NOT NULL
);

-- GOOD: Auto-generated ID as PK
CREATE TABLE day_4.good_customers (
    customer_id SERIAL PRIMARY KEY,
    email VARCHAR(150) UNIQUE NOT NULL,
    name VARCHAR(100) NOT NULL
);
-- Why surrogate is better:
-- 1. Email can change; SERIAL never changes
-- 2. INT is 4 bytes; email is 150 bytes (slower joins)
-- 3. No cascading updates needed
```

Hard: Composite Primary Key — Attendance System

HARD

SCENARIO: TCS attendance system: one record per employee per day. Neither emp_id alone nor date alone is unique. But employee 101 on Jan 15 can only appear ONCE.

TASK: Create day_4.attendance with PRIMARY KEY (emp_id, att_date). Insert emp 101 on Jan 15 and Jan 16 (works). Insert emp 102 on Jan 15 (works). Try emp 101 on Jan 15 again (blocked).

HINT: Composite PK = the COMBINATION must be unique. Same employee, different dates = OK. Same date, different employees = OK.

Answer

✅ ANSWER

```
CREATE TABLE day_4.attendance (
    emp_id INT NOT NULL,
    att_date DATE NOT NULL,
    status VARCHAR(10) NOT NULL DEFAULT 'present'
      CHECK (status IN ('present','absent','half_day','leave')),
    PRIMARY KEY (emp_id, att_date)
);

INSERT INTO day_4.attendance (emp_id, att_date, status)
VALUES (101, '2025-01-15', 'present');  -- OK
INSERT INTO day_4.attendance (emp_id, att_date, status)
VALUES (101, '2025-01-16', 'present');  -- OK (diff date)
INSERT INTO day_4.attendance (emp_id, att_date, status)
VALUES (102, '2025-01-15', 'half_day'); -- OK (diff emp)

-- BLOCKED: same emp + same date
INSERT INTO day_4.attendance (emp_id, att_date, status)
VALUES (101, '2025-01-15', 'absent');
```

Crazy: UUID Primary Key for Distributed Systems

CRAZY

SCENARIO: Ola runs multiple servers generating ride records simultaneously. SERIAL would cause ID conflicts between servers. UUID solves this — each server generates globally unique IDs independently.

TASK: Create day_4.rides with ride_id UUID PRIMARY KEY DEFAULT gen_random_uuid(). Insert two rides without specifying IDs. Verify each got a unique UUID.

HINT: gen_random_uuid() creates IDs like '550e8400-e29b-41d4-a716-446655440000'. Collision chance: 1 in 2^122.

Answer

✅ ANSWER

```
CREATE TABLE day_4.rides (
    ride_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    rider_name VARCHAR(100) NOT NULL,
    pickup_city VARCHAR(50) NOT NULL,
    fare NUMERIC(8,2) NOT NULL CHECK (fare > 0),
    created_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

INSERT INTO day_4.rides (rider_name, pickup_city, fare)
VALUES ('Deepika', 'Mumbai', 1250.00);
INSERT INTO day_4.rides (rider_name, pickup_city, fare)
VALUES ('Arjun', 'Delhi', 350.00);
-- Each gets a unique UUID automatically
```

KEY TAKEAWAYS

PRIMARY KEY

Every table MUST have a PK. SERIAL for simple apps, UUID for distributed systems, composite for junction tables. Never use real-world data (email, phone) as PK — it changes.

INTERVIEW

Natural vs Surrogate Keys

Q: 'Natural Key (email) or Surrogate Key (SERIAL)?'

A: Almost always Surrogate. Natural keys change, are long, and leak business data. SERIAL/UUID are immutable, compact, and meaningless — perfect for stable identifiers. Use UNIQUE for natural identifiers.

STORY

The Myntra Phantom Order

A developer manually inserts an order: customer_id = 99999, product_id = 88888. Neither actually exists. The INSERT succeeds. No error.

Two weeks later, a finance report crashes — these IDs lead nowhere. These are ORPHANED RECORDS — children without parents.

With a FOREIGN KEY constraint, the INSERT would be rejected INSTANTLY: 'customer_id 99999 does not exist in the customers table.' A FK is like a marriage certificate — both parties must exist before the registrar approves the link.

DEFINITION

FOREIGN KEY

Technical:
A FOREIGN KEY (FK) is a column in one table that references the PRIMARY KEY (or UNIQUE column) of another table. It enforces referential integrity — the child can only contain values that already exist in the parent. Any INSERT or UPDATE violating this is rejected.

Layman's:
Every student ID card has a 'Class' field like '10-B'. A FOREIGN KEY ensures '10-B' actually exists in the school's class list. If someone writes '10-Z', the system rejects it because that class does not exist. The student's class field is 'foreign' — it belongs to the classes table.

SYNTAX: FOREIGN KEY (1/2)

```
-- Method 1: Column-level (inline)
CREATE TABLE child_table (
    child_id SERIAL PRIMARY KEY,
    parent_id INT REFERENCES parent_table(parent_id)
);

-- Method 2: Table-level (explicit)
CREATE TABLE child_table (
    child_id SERIAL PRIMARY KEY,
    parent_id INT NOT NULL,
    FOREIGN KEY (parent_id) REFERENCES parent_table(parent_id)
);

-- Method 3: Named constraint
CREATE TABLE child_table (
    child_id SERIAL PRIMARY KEY,
    parent_id INT NOT NULL,
    CONSTRAINT fk_parent
        FOREIGN KEY (parent_id)
        REFERENCES parent_table(parent_id)
```

SYNTAX: FOREIGN KEY (2/2)

```
);
```

Easy: Connecting Orders to Customers

EASY

SCENARIO: RetailMart needs an orders table. Every order must belong to a real customer — no ghost orders pointing to non-existent people.

TASK: Create day_4.rm_customers (customer_id SERIAL PK, full_name NOT NULL, email UNIQUE NOT NULL). Then create day_4.rm_orders with a customer_id FK. Insert a customer, create an order for them, then try inserting an order for customer_id = 999.

HINT: The child table (orders) must reference an existing parent (customers). Insert parent first, then child.

Answer (1/2)

✅ ANSWER

```
CREATE TABLE day_4.rm_customers (
    customer_id SERIAL PRIMARY KEY,
    full_name VARCHAR(100) NOT NULL,
    email VARCHAR(150) UNIQUE NOT NULL
);

CREATE TABLE day_4.rm_orders (
    order_id SERIAL PRIMARY KEY,
    customer_id INT NOT NULL
        REFERENCES day_4.rm_customers(customer_id),
    order_total NUMERIC(10,2) NOT NULL CHECK (order_total > 0),
    ordered_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

INSERT INTO day_4.rm_customers (full_name, email)
VALUES ('Meera Reddy', 'meera@gmail.com');

INSERT INTO day_4.rm_orders (customer_id, order_total)
VALUES (1, 2499.00);  -- Works! Customer 1 exists.

INSERT INTO day_4.rm_orders (customer_id, order_total)
VALUES (999, 500.00);
```

Answer (2/2)

✅ ANSWER

```
-- ERROR: Key (customer_id)=(999) is not present
-- in table "rm_customers"
```

Medium: Multi-Table FK — Orders Reference Customers AND Stores

MEDIUM

SCENARIO: RetailMart orders are placed at physical stores. Each order must link to both a valid customer AND a valid store. A table can have multiple foreign keys.

TASK: Create day_4.rm_stores (store_id SERIAL PK, store_name NOT NULL, city NOT NULL). Create day_4.rm_store_orders with FKs to both rm_customers and rm_stores. Insert valid parents, then a valid order, then try store_id = 99.

HINT: Each FK is validated independently. ALL must pass for the INSERT to succeed.

Answer (1/2)

✅ ANSWER

```
CREATE TABLE day_4.rm_stores (
    store_id SERIAL PRIMARY KEY,
    store_name VARCHAR(100) NOT NULL,
    city VARCHAR(50) NOT NULL
);

CREATE TABLE day_4.rm_store_orders (
    order_id SERIAL PRIMARY KEY,
    customer_id INT NOT NULL
        REFERENCES day_4.rm_customers(customer_id),
    store_id INT NOT NULL
        REFERENCES day_4.rm_stores(store_id),
    order_total NUMERIC(10,2) NOT NULL CHECK (order_total > 0),
    ordered_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

INSERT INTO day_4.rm_stores (store_name, city)
VALUES ('RetailMart Koramangala', 'Bangalore');

INSERT INTO day_4.rm_store_orders
    (customer_id, store_id, order_total)
VALUES (1, 1, 3500.00);  -- Works!
```

Answer (2/2)

✅ ANSWER

```
INSERT INTO day_4.rm_store_orders
    (customer_id, store_id, order_total)
VALUES (1, 99, 1000.00);
-- ERROR: FK constraint violation on store_id
```

Hard: Self-Referencing FK — Employees Reporting to Employees

HARD

SCENARIO: At Google India, every employee has a manager — but the manager is also an employee in the same table. The CEO has no manager (NULL). How do you model a hierarchy within a single table?

TASK: Create day_4.org_employees where manager_id references emp_id in the same table. Insert a CEO (no manager), a VP reporting to the CEO, an engineer reporting to the VP. Then try inserting someone with manager_id = 500.

HINT: The top of the hierarchy has manager_id = NULL. The FK column must allow NULL (no NOT NULL constraint).

Answer (1/2)

✅ ANSWER

```
CREATE TABLE day_4.org_employees (
    emp_id SERIAL PRIMARY KEY,
    emp_name VARCHAR(100) NOT NULL,
    designation VARCHAR(50) NOT NULL,
    manager_id INT
        REFERENCES day_4.org_employees(emp_id),
    hire_date DATE NOT NULL DEFAULT CURRENT_DATE
);

-- CEO: no manager
INSERT INTO day_4.org_employees (emp_name, designation)
VALUES ('Sundar Pichai', 'CEO');

-- VP reports to CEO (emp_id = 1)
INSERT INTO day_4.org_employees
    (emp_name, designation, manager_id)
VALUES ('Prabhakar Raghavan', 'VP Search', 1);

-- Engineer reports to VP (emp_id = 2)
INSERT INTO day_4.org_employees
    (emp_name, designation, manager_id)
VALUES ('Ravi Kumar', 'Senior Engineer', 2);
```

Answer (2/2)

✅ ANSWER

```
-- FAILS: manager 500 does not exist
INSERT INTO day_4.org_employees
    (emp_name, designation, manager_id)
VALUES ('Ghost', 'Intern', 500);
-- ERROR: FK constraint violation
```

Crazy: Complete Mini-Schema with Interlinked FKs

CRAZY

SCENARIO: RetailMart needs categories, brands, products, and order_items. Products reference both categories and brands. Order items reference both orders and products. This is a 4-level FK dependency chain.

TASK: Create all 4 tables with proper FKs. Insert data following the correct dependency order: categories → brands → products → order_items. What happens if you try to insert an order_item before the product exists?

HINT: The insertion order MUST follow the FK chain. Category and Brand are parents of Product. Order and Product are parents of Order Item.

Answer (1/2)

✅ ANSWER

```
CREATE TABLE day_4.rm_categories (
    category_id SERIAL PRIMARY KEY,
    category_name VARCHAR(50) UNIQUE NOT NULL
);
CREATE TABLE day_4.rm_brands (
    brand_id SERIAL PRIMARY KEY,
    brand_name VARCHAR(100) UNIQUE NOT NULL
);
CREATE TABLE day_4.rm_products (
    product_id SERIAL PRIMARY KEY,
    product_name VARCHAR(200) NOT NULL,
    category_id INT NOT NULL
        REFERENCES day_4.rm_categories(category_id),
    brand_id INT NOT NULL
        REFERENCES day_4.rm_brands(brand_id),
    price NUMERIC(10,2) NOT NULL CHECK (price > 0),
    stock_qty INT NOT NULL DEFAULT 0 CHECK (stock_qty >= 0)
);
CREATE TABLE day_4.rm_order_items (
    item_id SERIAL PRIMARY KEY,
    order_id INT NOT NULL
        REFERENCES day_4.rm_orders(order_id),
```

Answer (2/2)

✅ ANSWER

```
    product_id INT NOT NULL
        REFERENCES day_4.rm_products(product_id),
    quantity INT NOT NULL CHECK (quantity > 0),
    unit_price NUMERIC(10,2) NOT NULL CHECK (unit_price > 0)
);

INSERT INTO day_4.rm_categories (category_name)
VALUES ('Electronics');
INSERT INTO day_4.rm_brands (brand_name)
VALUES ('Samsung');
INSERT INTO day_4.rm_products
    (product_name, category_id, brand_id, price, stock_qty)
VALUES ('Galaxy S24 Ultra', 1, 1, 134999.00, 50);
INSERT INTO day_4.rm_order_items
    (order_id, product_id, quantity, unit_price)
VALUES (1, 1, 1, 134999.00);
```

KEY TAKEAWAYS

FOREIGN KEY

A FK ensures every child value exists as a PK in the parent table. Parent created BEFORE child. Data inserted into parent BEFORE child. Foreign keys are the foundation of 'Relational' in Relational Database.

COMMON MISTAKE

Mismatched Data Types on FK

The Mistake:

Parent PK is BIGINT but child FK column is INT. PostgreSQL refuses to create the constraint.

The Fix:

FK column and referenced PK MUST have the exact same data type. SERIAL parent → INT child. UUID parent → UUID child.

INTERVIEW

Orphaned Records

Q: 'What is an orphaned record and how do you prevent it?'

A: An orphaned record is a child row whose FK value does not match any parent row. Example: order with customer_id = 500 when no customer 500 exists. Prevented entirely by defining FOREIGN KEY constraints — the database rejects any insert that would create an orphan.

CHALLENGE

MINI CHALLENGE: Foreign Keys

1.: Create day_4.departments (dept_id SERIAL PK, dept_name UNIQUE NOT NULL) and day_4.staff (staff_id SERIAL PK, staff_name NOT NULL, dept_id INT NOT NULL REFERENCES day_4.departments).

2.: Insert department 'Engineering'. Insert a staff member in that department.

3.: Try inserting a staff member with dept_id = 999 and observe the FK error.

STORY

The Tech Mahindra Department Shutdown

Tech Mahindra shuts down 'Legacy Systems'. HR deletes the department row. But 200 employees point to it.

RESTRICT: Database BLOCKS the delete. 'Reassign employees first.' CASCADE: Deletes department AND all 200 employees. SET NULL: Deletes department, sets employees' dept_id to NULL — they stay, unassigned.

Each option is valid in different situations. The choice is made ONCE — when you define the FOREIGN KEY.

DEFINITION

ON DELETE Actions

Technical:
ON DELETE is a clause on a FOREIGN KEY that specifies what happens to child rows when the parent is deleted. Options: RESTRICT (block), CASCADE (delete children), SET NULL (null-ify FK), SET DEFAULT (set FK to default), NO ACTION (like RESTRICT, checked at transaction end).

Layman's:
A father wants to leave the country. RESTRICT: 'You cannot leave until children are handled.' CASCADE: 'You leave, children are removed too.' SET NULL: 'You leave, children stay but father field becomes blank.'

SYNTAX: ON DELETE Actions

```
-- RESTRICT (default — blocks parent deletion)
FOREIGN KEY (parent_id) REFERENCES parent(id)
    ON DELETE RESTRICT;

-- CASCADE (auto-delete children)
FOREIGN KEY (parent_id) REFERENCES parent(id)
    ON DELETE CASCADE;

-- SET NULL (unlink children)
FOREIGN KEY (parent_id) REFERENCES parent(id)
    ON DELETE SET NULL;

-- Can combine with ON UPDATE:
FOREIGN KEY (parent_id) REFERENCES parent(id)
    ON DELETE CASCADE ON UPDATE CASCADE;
```

Easy: ON DELETE RESTRICT — The Safe Default

EASY

SCENARIO: RetailMart has departments and employees. The company should NOT be able to delete a department while employees still belong to it — that would leave people orphaned.

TASK: Create day_4.departments and day_4.dept_employees with ON DELETE RESTRICT. Insert a department, add an employee, then try to DELETE the department.

HINT: RESTRICT is the default if you do not specify ON DELETE at all.

Answer

✅ ANSWER

```
CREATE TABLE day_4.departments (
    dept_id SERIAL PRIMARY KEY,
    dept_name VARCHAR(50) UNIQUE NOT NULL
);
CREATE TABLE day_4.dept_employees (
    emp_id SERIAL PRIMARY KEY,
    emp_name VARCHAR(100) NOT NULL,
    dept_id INT NOT NULL
        REFERENCES day_4.departments(dept_id)
        ON DELETE RESTRICT
);

INSERT INTO day_4.departments (dept_name)
VALUES ('Engineering');
INSERT INTO day_4.dept_employees (emp_name, dept_id)
VALUES ('Arun Nair', 1);

DELETE FROM day_4.departments WHERE dept_id = 1;
-- ERROR: update or delete on "departments"
-- violates FK constraint
-- Key (dept_id)=(1) is still referenced
```

Medium: ON DELETE CASCADE — Auto-Delete Children

MEDIUM

SCENARIO: A blogging platform stores posts and comments. When a post is deleted, its comments are meaningless — they should be automatically removed too.

TASK: Create day_4.blog_posts and day_4.post_comments with ON DELETE CASCADE. Insert a post with 3 comments. Delete the post. Verify all comments are gone.

HINT: CASCADE is perfect for dependent data that has no meaning without its parent.

Answer (1/2)

✅ ANSWER

```
CREATE TABLE day_4.blog_posts (
    post_id SERIAL PRIMARY KEY,
    title VARCHAR(200) NOT NULL,
    content TEXT NOT NULL
);
CREATE TABLE day_4.post_comments (
    comment_id SERIAL PRIMARY KEY,
    post_id INT NOT NULL
        REFERENCES day_4.blog_posts(post_id)
        ON DELETE CASCADE,
    comment_text TEXT NOT NULL,
    commented_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

INSERT INTO day_4.blog_posts (title, content)
VALUES ('Learn SQL', 'Constraints are important...');
INSERT INTO day_4.post_comments (post_id, comment_text)
VALUES (1, 'Great article!'),
       (1, 'Very helpful!'),
       (1, 'Bookmarked.');

DELETE FROM day_4.blog_posts WHERE post_id = 1;
```

Answer (2/2)

✅ ANSWER

```
SELECT count(*) FROM day_4.post_comments; -- 0 rows!
-- All 3 comments auto-deleted with the post
```

Hard: ON DELETE SET NULL — Keep Children, Break the Link

HARD

SCENARIO: At a company, each team member has a manager. When Kavitha (manager) leaves the company, her team members should NOT be fired — they should stay but become temporarily unassigned (manager_id = NULL).

TASK: Create day_4.managers and day_4.team_members with ON DELETE SET NULL. Insert a manager with 3 team members. Delete the manager. Verify members still exist with manager_id = NULL.

HINT: The FK column must ALLOW NULL (no NOT NULL constraint) for SET NULL to work.

Answer

✅ ANSWER

```
CREATE TABLE day_4.managers (
    manager_id SERIAL PRIMARY KEY,
    manager_name VARCHAR(100) NOT NULL
);
CREATE TABLE day_4.team_members (
    member_id SERIAL PRIMARY KEY,
    member_name VARCHAR(100) NOT NULL,
    manager_id INT  -- NO 'NOT NULL' here!
        REFERENCES day_4.managers(manager_id)
        ON DELETE SET NULL
);

INSERT INTO day_4.managers (manager_name)
VALUES ('Kavitha Iyer');
INSERT INTO day_4.team_members (member_name, manager_id)
VALUES ('Rohit', 1), ('Sneha', 1), ('Amit', 1);

DELETE FROM day_4.managers WHERE manager_id = 1;
SELECT * FROM day_4.team_members;
-- All 3 members still exist, manager_id = NULL
-- HR can later assign a new manager
```

Crazy: Choosing the Right Policy — RetailMart Analysis

CRAZY

SCENARIO: RetailMart has three deletion scenarios. (1) Deleting a CATEGORY — should products disappear? (2) Deleting an ORDER — should order_items vanish? (3) Deleting a CUSTOMER — should their financial order records vanish?

TASK: For each scenario, choose the right ON DELETE policy (RESTRICT, CASCADE, or SET NULL) and create the tables accordingly. Explain WHY each choice is correct.

HINT: Ask yourself — 'If the parent disappears, does the child still make sense on its own?'

Answer (1/2)

✅ ANSWER

```
-- SCENARIO 1: Category deleted → Products survive
-- Use SET NULL: product stays, becomes uncategorized
CREATE TABLE day_4.safe_products (
    product_id SERIAL PRIMARY KEY,
    product_name VARCHAR(200) NOT NULL,
    category_id INT  -- allows NULL
        REFERENCES day_4.rm_categories(category_id)
        ON DELETE SET NULL
);

-- SCENARIO 2: Order deleted → Order items are useless
-- Use CASCADE: items die with the order
CREATE TABLE day_4.safe_order_items (
    item_id SERIAL PRIMARY KEY,
    order_id INT NOT NULL
        REFERENCES day_4.rm_orders(order_id)
        ON DELETE CASCADE,
    product_id INT NOT NULL
        REFERENCES day_4.rm_products(product_id)
        ON DELETE RESTRICT,
    quantity INT NOT NULL CHECK (quantity > 0)
);
```

Answer (2/2)

✅ ANSWER

```
-- SCENARIO 3: Customer deleted → Orders are financial records
-- Use RESTRICT. NEVER delete financial records.
-- Force manual archival first (soft-delete).
```

KEY TAKEAWAYS

Cascading Actions

RESTRICT: Safest — blocks parent deletion if children exist. Use for critical data. CASCADE: Auto-deletes children — use for disposable dependent data. SET NULL: Keeps children, removes link — use for reassignable relationships. Always choose based on business rules, never convenience.

COMMON MISTAKE

CASCADE on Financial Records

The Mistake:
Using ON DELETE CASCADE between customers and orders. Deleting a customer wipes ALL their financial order records — breaking auditing and tax compliance.

The Fix:
NEVER use CASCADE on financial/legal/audit relationships. Use RESTRICT + soft-delete (is_active = FALSE) in the application layer instead.

PRO TIP

Soft Delete — The Industry Standard

At Razorpay, Paytm, and Groww, you will almost NEVER see CASCADE on core financial tables. The standard: RESTRICT on all critical FKs + soft-delete in the app layer (SET is_active = FALSE instead of DELETE). CASCADE is only for truly disposable data like session logs or temp carts.

INTERVIEW

CASCADE vs RESTRICT

Q: 'When would you use CASCADE vs RESTRICT?'

A: CASCADE for dependent data with no standalone meaning — comments on a deleted post, items in a cancelled cart. RESTRICT for data that must be preserved — orders belonging to a closing customer account. In fintech, RESTRICT is the safe default. One CASCADE DELETE can silently wipe thousands of child rows.

CHALLENGE

MINI CHALLENGE: Cascading Actions

1.: Create day_4.projects (PK) and day_4.tasks (FK to projects ON DELETE CASCADE).

2.: Insert a project with 3 tasks. Delete the project. Verify all tasks are gone.

3.: Think: Would you use CASCADE or RESTRICT if the child table was 'invoices'?

STORY

The Empty Restaurant Analogy

You spent 3 days designing the perfect restaurant. Kitchen layout done (CREATE TABLE). Equipment specs finalized (data types). Safety rules posted (constraints). Fire exits connected (foreign keys).

But there is no food. No customers. The restaurant is structurally perfect but operationally useless. DML is where you finally START USING it — INSERT stocks the pantry, UPDATE changes today's special, DELETE removes expired items, TRUNCATE resets everything fresh.

STORY

The First Day at Meesho Warehouse

When Meesho opens a new warehouse, operations INSERT every product into the inventory table. Without INSERT, the system shows zero products, delivery partners have nothing to pick, and customers see 'Out of Stock' everywhere.

INSERT is the most fundamental DML command. Every row in every database in the world got there because someone ran an INSERT.

DEFINITION

INSERT INTO

Technical:
INSERT INTO adds one or more new rows. Values must match column data types and satisfy all constraints (NOT NULL, UNIQUE, CHECK, FK). If any constraint fails, the entire INSERT is rejected.

Layman's:
INSERT is like filling out a new row in Excel — but with strict rules. Every required field must be filled. No duplicates where banned. Every reference must point to something real.

SYNTAX: INSERT INTO

```
-- Single row (specify columns)
INSERT INTO table_name (col1, col2, col3)
VALUES ('value1', 42, NOW());

-- Multiple rows at once
INSERT INTO table_name (col1, col2)
VALUES ('a', 1), ('b', 2), ('c', 3);

-- INSERT ... RETURNING (get auto-generated values)
INSERT INTO table_name (col1, col2)
VALUES ('value1', 42)
RETURNING id, created_at;
```

Easy: Inserting a Single Row with SERIAL

EASY

SCENARIO: A restaurant is adding items to their digital menu system. Each item gets an auto-generated ID from SERIAL. The menu tracks name, price, category, and availability.

TASK: Create day_4.menu_items with SERIAL PK, item_name NOT NULL, price CHECK > 0, category NOT NULL, is_available DEFAULT TRUE. Insert 'Butter Chicken' at Rs. 350 (Main Course) and 'Masala Dosa' at Rs. 120 (South Indian). Show the result.

HINT: You do NOT specify the SERIAL column — PostgreSQL auto-generates 1, 2, 3...

Answer

✅ ANSWER

```
CREATE TABLE day_4.menu_items (
    item_id SERIAL PRIMARY KEY,
    item_name VARCHAR(100) NOT NULL,
    price NUMERIC(8,2) NOT NULL CHECK (price > 0),
    category VARCHAR(50) NOT NULL,
    is_available BOOLEAN NOT NULL DEFAULT TRUE
);

INSERT INTO day_4.menu_items (item_name, price, category)
VALUES ('Butter Chicken', 350.00, 'Main Course');
INSERT INTO day_4.menu_items (item_name, price, category)
VALUES ('Masala Dosa', 120.00, 'South Indian');

SELECT * FROM day_4.menu_items;
-- 1 | Butter Chicken | 350.00 | Main Course  | true
-- 2 | Masala Dosa    | 120.00 | South Indian | true
```

Medium: Multi-Row INSERT and RETURNING

MEDIUM

SCENARIO: The restaurant just launched and needs to bulk-load 5 menu items at once. The manager also wants to see the auto-generated IDs immediately after inserting the flagship item (Biryani).

TASK: Insert 5 items in a single INSERT statement. Then insert 'Biryani' separately using RETURNING to see the generated item_id and is_available.

HINT: Multi-row INSERT uses VALUES ('a', 1), ('b', 2), ('c', 3). RETURNING is PostgreSQL-specific — not available in MySQL.

Answer

✅ ANSWER

```
INSERT INTO day_4.menu_items (item_name, price, category)
VALUES
    ('Paneer Tikka', 280.00, 'Starters'),
    ('Dal Makhani', 220.00, 'Main Course'),
    ('Gulab Jamun', 80.00, 'Desserts'),
    ('Mango Lassi', 90.00, 'Beverages'),
    ('Naan', 40.00, 'Breads');

INSERT INTO day_4.menu_items (item_name, price, category)
VALUES ('Biryani', 300.00, 'Main Course')
RETURNING item_id, item_name, is_available;
-- Returns: item_id=8, item_name=Biryani, is_available=true
```

Hard: INSERT with Constraint Violations

HARD

SCENARIO: An orders table has NOT NULL on customer_name, CHECK (amount > 0), and CHECK (status IN ('pending','confirmed','shipped','delivered')). A developer tries 3 invalid inserts that each violate a different constraint.

TASK: Create the constrained day_4.dml_orders table. Show 3 failing INSERTs — one missing customer_name, one with negative amount, one with invalid status. Then show the one valid INSERT that passes all checks.

HINT: Read the error messages — they tell you exactly WHICH constraint failed.

Answer (1/2)

✅ ANSWER

```
CREATE TABLE day_4.dml_orders (
    order_id SERIAL PRIMARY KEY,
    customer_name VARCHAR(100) NOT NULL,
    amount NUMERIC(10,2) NOT NULL CHECK (amount > 0),
    status VARCHAR(20) NOT NULL DEFAULT 'pending'
        CHECK (status IN
        ('pending','confirmed','shipped','delivered')),
    created_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

-- FAILS: NOT NULL (customer_name missing)
INSERT INTO day_4.dml_orders (amount)
VALUES (500.00);

-- FAILS: CHECK (negative amount)
INSERT INTO day_4.dml_orders (customer_name, amount)
VALUES ('Priya', -100.00);

-- FAILS: CHECK (invalid status 'processing')
INSERT INTO day_4.dml_orders
    (customer_name, amount, status)
VALUES ('Priya', 500.00, 'processing');
```

Answer (2/2)

✅ ANSWER

```
-- SUCCESS: all constraints pass
INSERT INTO day_4.dml_orders (customer_name, amount)
VALUES ('Priya Sharma', 2499.00);
-- status='pending', created_at=NOW() auto-filled
```

Crazy: INSERT into FK Chain — Order Matters

CRAZY

SCENARIO: RetailMart's 4-table FK chain: categories → brands → products → order_items. You need to populate the entire chain. If you try to insert an order_item before its product exists, the FK rejects it.

TASK: Insert data into all 4 tables in the correct FK dependency order. Demonstrate that skipping a step causes a FK violation.

HINT: Parent first, child second, grandchild third. The insertion order mirrors the CREATE TABLE order.

Answer (1/2)

✅ ANSWER

```
-- Step 1: Insert category (parent)
INSERT INTO day_4.rm_categories (category_name)
VALUES ('Fashion');

-- Step 2: Insert brand (parent)
INSERT INTO day_4.rm_brands (brand_name)
VALUES ('Zara');

-- Step 3: Product (child of category + brand)
INSERT INTO day_4.rm_products
    (product_name, category_id, brand_id, price, stock_qty)
VALUES ('Slim Fit Shirt', 2, 2, 2999.00, 100);

-- Step 4: Customer (parent for order)
INSERT INTO day_4.rm_customers (full_name, email)
VALUES ('Arjun Kapoor', 'arjun@gmail.com');

-- Step 5: Order (child of customer)
INSERT INTO day_4.rm_orders (customer_id, order_total)
VALUES (2, 2999.00);

-- Step 6: Order item (grandchild)
```

Answer (2/2)

✅ ANSWER

```
INSERT INTO day_4.rm_order_items
    (order_id, product_id, quantity, unit_price)
VALUES (2, 2, 1, 2999.00);
-- If Step 6 before Step 5: FK violation!
```

KEY TAKEAWAYS

INSERT INTO

INSERT adds new rows. Always specify column names explicitly. Use multi-row INSERT for bulk loading. Use RETURNING for auto-generated values. Respect FK order: parent before child. Every constraint is checked on INSERT.

PRO TIP

INSERT … RETURNING — PostgreSQL's Secret Weapon

In app code, you always need the auto-generated ID after inserting. MySQL: INSERT then SELECT LAST_INSERT_ID() (two statements). PostgreSQL: add RETURNING id — one statement, faster, no race conditions.

STORY

The Flipkart Big Billion Day Price Error

During Big Billion Day, a developer writes: UPDATE products SET price = price * 0.5 — forgetting the WHERE clause. EVERY product gets 50% off. PS5 at Rs. 24,999. iPhone at Rs. 64,999. Flipkart loses crores in 15 minutes.

UPDATE without WHERE is the most dangerous SQL statement. It changes EVERY row. Always, always include WHERE unless you genuinely want to modify all rows.

DEFINITION

UPDATE

Technical:

UPDATE modifies existing rows. SET specifies new values, WHERE filters which rows change. Without WHERE, ALL rows are updated. The update must satisfy all constraints — violated CHECK or UNIQUE rejects the entire UPDATE.

Layman's:

UPDATE is like 'Find and Replace' in Excel but with power to change specific columns in specific rows. Forget to specify WHICH rows? It changes ALL of them.

SYNTAX: UPDATE

```sql
-- Basic UPDATE with WHERE
UPDATE table_name
SET column1 = new_value
WHERE condition;

-- UPDATE with expression
UPDATE table_name
SET price = price * 0.9  -- 10% discount
WHERE category = 'Electronics';

-- UPDATE with RETURNING
UPDATE table_name
SET status = 'shipped'
WHERE order_id = 42
RETURNING order_id, status;
```

Easy: Update a Single Row by Primary Key

EASY

SCENARIO: The restaurant decides to increase the price of Butter Chicken from Rs. 350 to Rs. 399. They also need to mark Naan as unavailable because they ran out of flour.

TASK: Update the price of item_id = 1 to 399.00. Then set is_available = FALSE for item_id = 5. Verify both changes with SELECT.

HINT: Always update by PRIMARY KEY to ensure exactly one row is affected.

Answer

✅ ANSWER

```
UPDATE day_4.menu_items
SET price = 399.00
WHERE item_id = 1;

SELECT * FROM day_4.menu_items WHERE item_id = 1;
-- 1 | Butter Chicken | 399.00 | Main Course | true

UPDATE day_4.menu_items
SET is_available = FALSE
WHERE item_id = 5;
```

Medium: Update Multiple Rows with a Condition

MEDIUM

SCENARIO: The restaurant runs a monsoon sale — 10% off all Main Course items. They also want to tag all items under Rs. 100 as 'Budget Friendly'. And they want to see which bread items were affected by a Rs. 20 price increase.

TASK: Apply 10% discount to category 'Main Course'. Add a tag column and set it to 'Budget Friendly' for price < 100. Increase bread prices by Rs. 20 using RETURNING.

HINT: Use SET price = price * 0.9 for percentage discounts. RETURNING shows affected rows.

Answer

✅ ANSWER

```
-- 10% discount on Main Course
UPDATE day_4.menu_items
SET price = price * 0.9
WHERE category = 'Main Course';

-- Add tag column, mark budget items
ALTER TABLE day_4.menu_items
ADD COLUMN tag VARCHAR(30);

UPDATE day_4.menu_items
SET tag = 'Budget Friendly'
WHERE price < 100;

-- Increase bread prices, see affected rows
UPDATE day_4.menu_items
SET price = price + 20
WHERE category = 'Breads'
RETURNING item_id, item_name, price;
```

Hard: UPDATE that Triggers Constraint Violations

HARD

SCENARIO: A developer tries 3 invalid updates: setting a negative price (CHECK violation), duplicating an email (UNIQUE violation), and changing an order's customer_id to a non-existent customer (FK violation).

TASK: Show all 3 failing UPDATEs with their error messages. Explain why constraints protect data during UPDATE, not just INSERT.

HINT: Constraints are enforced on EVERY data change — INSERT and UPDATE both.

Answer

✅ ANSWER

```
-- FAILS: CHECK (negative price)
UPDATE day_4.menu_items
SET price = -50.00 WHERE item_id = 1;
-- ERROR: violates check constraint

-- FAILS: UNIQUE (duplicate email)
UPDATE day_4.rm_customers
SET email = 'meera@gmail.com'
WHERE customer_id = 2;
-- ERROR: duplicate key value violates unique

-- FAILS: FK (non-existent customer)
UPDATE day_4.rm_orders
SET customer_id = 9999 WHERE order_id = 1;
-- ERROR: FK constraint violation
```

Crazy: UPDATE with Multiple Columns and Calculated Values

CRAZY

SCENARIO: RetailMart needs to: (1) Ship a pending order and record the timestamp. (2) Apply a 15% price increase to all in-stock products, but cap the max at Rs. 50,000. (3) Confirm all remaining pending orders.

TASK: Write 3 UPDATE statements using SET with multiple columns, expressions (price * 1.15), functions (NOW(), LEAST()), and RETURNING.

HINT: LEAST(a, b) returns the smaller value — use it to cap prices.

Answer

✅ ANSWER

```
-- Add shipped_at column
ALTER TABLE day_4.dml_orders
ADD COLUMN shipped_at TIMESTAMPTZ;

-- Ship order #1 and record timestamp
UPDATE day_4.dml_orders
SET status = 'shipped', shipped_at = NOW()
WHERE order_id = 1 AND status = 'pending'
RETURNING order_id, status, shipped_at;

-- 15% price increase, capped at 50000
UPDATE day_4.rm_products
SET price = LEAST(price * 1.15, 50000.00)
WHERE stock_qty > 0;

-- Confirm all pending orders
UPDATE day_4.dml_orders
SET status = 'confirmed'
WHERE status = 'pending'
RETURNING *;
```

KEY TAKEAWAYS

UPDATE

UPDATE modifies existing rows. ALWAYS include WHERE. Constraints are enforced on UPDATE too. Use RETURNING to see what changed. Pro tip: run your WHERE in a SELECT first to preview which rows will be affected.

COMMON MISTAKE

UPDATE Without WHERE — The Career-Ender

The Mistake:

UPDATE products SET price = 0; — No WHERE clause. Every product now costs Rs. 0. Revenue = zero.

The Fix:

Write WHERE FIRST. Better yet, test with SELECT: SELECT * FROM products WHERE category = 'Electronics'; — if the rows look right, change SELECT to UPDATE … SET.

STORY

The Ola Driver Account Purge

Ola wants to deactivate drivers with zero rides in 6 months. A developer writes: DELETE FROM drivers; — forgetting WHERE. ALL 5 lakh driver records vanish. Active drivers, top-rated drivers, everyone.

The app shows 'No drivers available' across India. It takes 4 hours to restore from backup. DELETE without WHERE destroys data. UPDATE without WHERE corrupts it. Both are equally dangerous.

DEFINITION

DELETE

Technical:
DELETE FROM removes rows. WHERE filters which rows are deleted. Without WHERE, ALL rows are removed (table structure remains). DELETE respects FK constraints — you cannot delete a parent row if children reference it (unless CASCADE). DELETE is logged and rollback-able inside a transaction.

Layman's:
DELETE removes specific rows from an Excel spreadsheet. The spreadsheet still exists with headers. If you forget to specify which rows, ALL data vanishes — but the empty spreadsheet remains.

SYNTAX: DELETE

```
-- Delete specific rows
DELETE FROM table_name WHERE condition;

-- Delete ALL rows (dangerous!)
DELETE FROM table_name;

-- Delete with RETURNING
DELETE FROM table_name
WHERE condition
RETURNING *;
```

Easy: Delete a Single Row by Primary Key

EASY

SCENARIO: The restaurant is removing Gulab Jamun from the menu because the chef quit. They need to delete just this one item by its ID.

TASK: Delete item_id = 5 from day_4.menu_items. Verify it is gone with SELECT. Confirm the table and other rows still exist.

HINT: Delete by PRIMARY KEY to ensure exactly one row is removed.

Answer

✅ ANSWER

```
DELETE FROM day_4.menu_items WHERE item_id = 5;

SELECT * FROM day_4.menu_items WHERE item_id = 5;
-- 0 rows (gone!)

SELECT count(*) FROM day_4.menu_items;
-- Count minus 1 — other rows untouched
```

Medium: Delete Multiple Rows with RETURNING

MEDIUM

SCENARIO: The restaurant wants to remove all unavailable items from the menu. They also want to clear out all beverages — but first, they want to see exactly which items will be removed before committing.

TASK: Delete all rows where is_available = FALSE. Then delete all 'Beverages' items using RETURNING to see what was removed. Show the SELECT-first preview technique.

HINT: Always preview with SELECT before DELETE. RETURNING gives one last look at deleted data.

Answer

✅ ANSWER

```
-- Remove all unavailable items
DELETE FROM day_4.menu_items
WHERE is_available = FALSE;

-- Preview first! What will be deleted?
SELECT * FROM day_4.menu_items
WHERE category = 'Beverages';
-- Review: are these the right rows?

-- If yes, delete with RETURNING
DELETE FROM day_4.menu_items
WHERE category = 'Beverages'
RETURNING item_id, item_name, price;
-- Shows exactly which rows were removed
```

Hard: DELETE Blocked by FOREIGN KEY — RESTRICT in Action

HARD

SCENARIO: An admin tries to delete a customer who has existing orders. The FK with RESTRICT blocks this to prevent orphaned order records. What are the alternatives?

TASK: Try DELETE FROM day_4.rm_customers WHERE customer_id = 1. Observe the FK error. Show 3 alternatives: delete child rows first (if CASCADE), reassign orders, or soft-delete the customer.

HINT: Soft-delete (UPDATE is_active = FALSE) is the industry standard for critical data.

Answer

✅ ANSWER

```
DELETE FROM day_4.rm_customers
WHERE customer_id = 1;
-- ERROR: violates FK constraint on "rm_orders"
-- Key (customer_id)=(1) is still referenced

-- Option A: Delete orders first (if CASCADE)
-- Option B: Reassign orders to another customer
-- Option C: Soft-delete (recommended)
UPDATE day_4.rm_customers
SET email = 'DELETED_' || email
WHERE customer_id = 1;
-- Customer marked deleted, orders preserved
```

Crazy: DELETE with CASCADE — Automatic Chain Cleanup

CRAZY

SCENARIO: A blog post with 3 comments has ON DELETE CASCADE. When the post is deleted, what happens to the comments? The developer needs to prove that CASCADE auto-deletes dependent data.

TASK: Insert a new blog post with 2 comments. Check comment count. Delete the post. Verify all comments are automatically gone. Explain when this is dangerous.

HINT: One DELETE on a parent can trigger cascading deletions across MULTIPLE child tables. This is powerful but dangerous for financial data.

Answer

✅ ANSWER

```
INSERT INTO day_4.blog_posts (title, content)
VALUES ('SQL Tips', 'Always use constraints!');
INSERT INTO day_4.post_comments (post_id, comment_text)
VALUES (2, 'Agreed!'), (2, 'This saved me once');

SELECT count(*) FROM day_4.post_comments;  -- 2

DELETE FROM day_4.blog_posts WHERE post_id = 2;

SELECT count(*) FROM day_4.post_comments;  -- 0!
-- Both comments auto-deleted with the post
-- WARNING: Never use CASCADE on financial tables!
```

KEY TAKEAWAYS

DELETE

DELETE removes rows. ALWAYS include WHERE. Respects FK constraints (RESTRICT blocks, CASCADE propagates). Use RETURNING to see removed data. Test with SELECT first. For critical data, prefer soft-delete (UPDATE is_active = FALSE) over hard DELETE.

COMMON MISTAKE

DELETE vs DROP — Know the Difference

The Mistake:

Confusing DELETE FROM t (removes rows, table remains) with DROP TABLE t (removes entire table structure). After DELETE, you can INSERT new rows. After DROP, the table is gone.

The Fix:

DELETE = remove data, keep structure. DROP = remove everything. TRUNCATE = remove all data fast, keep structure.

STORY

The Test Database Reset

Every morning at Razorpay, QA needs to reset 50 test tables with millions of rows. DELETE FROM each table takes 45 minutes — it scans every row, logs each deletion, fires triggers.

TRUNCATE TABLE takes 3 seconds. It simply deallocates the data pages — like tearing out all pages from a notebook instead of erasing each line one by one.

DEFINITION

TRUNCATE

Technical:

TRUNCATE removes all rows without scanning individual rows. Much faster than DELETE for large tables. Resets SERIAL counters back to 1. Technically DDL — auto-commits in some databases, but CAN be rolled back in PostgreSQL.

Layman's:

DELETE is erasing each answer on an exam paper one by one. TRUNCATE is throwing the paper in the bin and getting a fresh one. Both give a blank paper, but TRUNCATE is infinitely faster.

SYNTAX: TRUNCATE

```sql
-- Truncate a single table
TRUNCATE TABLE table_name;

-- Reset SERIAL counter too
TRUNCATE TABLE table_name RESTART IDENTITY;

-- Truncate with CASCADE (child tables too)
TRUNCATE TABLE parent_table CASCADE;
```

Easy: Basic TRUNCATE to Empty a Table

EASY

SCENARIO: The restaurant is renovating and wants to wipe the entire menu to start fresh. But the table structure, constraints, and indexes should remain intact.

TASK: Check the current row count of day_4.menu_items. Run TRUNCATE. Check count again (should be 0). Insert a fresh item to prove the table still works.

HINT: TRUNCATE empties the table but keeps the structure — like clearing a whiteboard.

Answer

✅ ANSWER

```
SELECT count(*) FROM day_4.menu_items;  -- e.g., 6

TRUNCATE TABLE day_4.menu_items;

SELECT count(*) FROM day_4.menu_items;  -- 0

-- Table still exists! Insert new rows.
INSERT INTO day_4.menu_items (item_name, price, category)
VALUES ('Fresh Start Thali', 199.00, 'Main Course');
```

Medium: TRUNCATE RESTART IDENTITY — Reset the Counter

MEDIUM

SCENARIO: After a regular TRUNCATE, the SERIAL counter continues from where it left off (e.g., item_id = 9). For a completely clean slate, you need RESTART IDENTITY to reset back to 1.

TASK: TRUNCATE day_4.menu_items (without RESTART). Insert an item and check its ID (will be high). Then TRUNCATE with RESTART IDENTITY. Insert again and verify ID starts at 1.

HINT: RESTART IDENTITY is essential for test database resets.

Answer

✅ ANSWER

```
-- Without RESTART: counter continues
TRUNCATE TABLE day_4.menu_items;
INSERT INTO day_4.menu_items (item_name, price, category)
VALUES ('Test Item', 100.00, 'Test');
SELECT * FROM day_4.menu_items;
-- item_id = 9 (continues from previous max)

-- With RESTART: counter resets to 1
TRUNCATE TABLE day_4.menu_items RESTART IDENTITY;
INSERT INTO day_4.menu_items (item_name, price, category)
VALUES ('Fresh Item', 100.00, 'Test');
SELECT * FROM day_4.menu_items;
-- item_id = 1 (counter reset!)
```

KEY TAKEAWAYS

TRUNCATE

TRUNCATE empties entire tables fast. Use for test resets, staging refreshes, bulk reloads. NOT a substitute for DELETE with WHERE — TRUNCATE always removes ALL rows. RESTART IDENTITY resets counters. In PostgreSQL, TRUNCATE can be rolled back in a transaction.

INTERVIEW

DROP vs TRUNCATE vs DELETE

Q: 'What is the difference between DROP, TRUNCATE, and DELETE?'

A: DROP removes the table AND all data — table ceases to exist. TRUNCATE removes ALL rows, keeps structure — fast, resets counters. DELETE removes specific rows (with WHERE) — slower but precise, logged, fires triggers. Memory trick: DROP = demolish building. TRUNCATE = empty all rooms. DELETE = move out specific furniture.

CHALLENGE

MINI CHALLENGE: DML Complete

1.: Create day_4.test_products: id SERIAL PK, name NOT NULL, price CHECK > 0, status DEFAULT 'active'.
2.: INSERT 5 products using multi-row INSERT with RETURNING.
3.: UPDATE one product's price. Try updating to negative — observe CHECK error.
4.: DELETE one product by id using RETURNING.
5.: TRUNCATE with RESTART IDENTITY. Insert a new row, verify id = 1.

KEY TAKEAWAYS

KEY TAKEAWAYS

1. NOT NULL: Reject empty values. Apply to every column where NULL would break logic.
2. DEFAULT: Auto-fill sensible values. Use NOW() for timestamps, FALSE for booleans, 0 for counters.
3. UNIQUE: No duplicates. Use for emails, phone, PAN. Multiple NULLs allowed in PostgreSQL.
4. CHECK: Custom validation. Ranges (BETWEEN), sets (IN), cross-column logic.
5. PRIMARY KEY: UNIQUE + NOT NULL. Use SERIAL or UUID. Composite PKs for junction tables.
6. FOREIGN KEY: Links child to parent. Enforces referential integrity — no orphans.
7. CASCADE: Auto-delete children. Only for disposable dependent data.
8. RESTRICT: Block parent deletion. Safe default for critical/financial data.
9. SET NULL: Keep children, break link. For reassignable relationships.
10. INSERT: Add rows. Specify columns. Use RETURNING. Respect FK order.
11. UPDATE: Modify rows. ALWAYS use WHERE. Test with SELECT first.
12. DELETE: Remove rows. ALWAYS use WHERE. Prefer soft-delete for critical data.
13. TRUNCATE: Empty tables fast. RESTART IDENTITY resets counters.

Constraints, Keys & DML

```
-- NOT NULL

 column_name DATA_TYPE NOT NULL
 ALTER TABLE t ALTER COLUMN c SET NOT NULL;
 ALTER TABLE t ALTER COLUMN c DROP NOT NULL;

-- DEFAULT

 column_name DATA_TYPE DEFAULT value
 column_name TIMESTAMPTZ DEFAULT NOW()
 column_name BOOLEAN DEFAULT TRUE
 column_name UUID DEFAULT gen_random_uuid()
 ALTER TABLE t ALTER COLUMN c SET DEFAULT value;

-- UNIQUE

 column_name DATA_TYPE UNIQUE
 UNIQUE (col_a, col_b)   -- multi-column
 ALTER TABLE t ADD CONSTRAINT name UNIQUE (col);
```

Constraints, Keys & DML

-- CHECK

```
column_name DATA_TYPE CHECK (condition)
CHECK (price >= 0)
CHECK (status IN ('a', 'b', 'c'))
CHECK (rating BETWEEN 1 AND 5)
CONSTRAINT name CHECK (end_date >= start_date)
```

-- PRIMARY KEY

```
column_name DATA_TYPE PRIMARY KEY
PRIMARY KEY (col_a, col_b)   -- composite
id SERIAL PRIMARY KEY
id UUID PRIMARY KEY DEFAULT gen_random_uuid()
```

-- FOREIGN KEY

```
col INT REFERENCES parent_table(parent_col)
FOREIGN KEY (col) REFERENCES parent_table(col)
CONSTRAINT fk_name FOREIGN KEY (col) REFERENCES parent(col)
```

Constraints, Keys & DML

```
-- ON DELETE Actions

 ON DELETE RESTRICT   -- block (safe default)
 ON DELETE CASCADE    -- auto-delete children
 ON DELETE SET NULL   -- keep children, break link

-- INSERT INTO

 INSERT INTO t (col1, col2) VALUES ('a', 1);
 INSERT INTO t (col1) VALUES ('a'), ('b'), ('c');
 INSERT INTO t (col1) VALUES ('a') RETURNING id;

-- UPDATE

 UPDATE t SET col = value WHERE condition;
 UPDATE t SET col = col * 1.1 WHERE id = 5;
 UPDATE t SET col = value RETURNING *;
```

Constraints, Keys & DML

```
-- DELETE

DELETE FROM t WHERE condition;
DELETE FROM t WHERE id = 5 RETURNING *;

-- TRUNCATE

TRUNCATE TABLE t;
TRUNCATE TABLE t RESTART IDENTITY;
TRUNCATE TABLE t CASCADE;
```

CHALLENGE

Constraints & DML Take-Home Challenge: Build a Constrained HR System (1/2)

Build the complete, bulletproof foundation for an Employee Management system.

Part A: Schema & Tables

1.: CREATE SCHEMA IF NOT EXISTS hr_practice;

2.: hr_practice.departments: dept_id SERIAL PK, dept_name VARCHAR(50) UNIQUE NOT NULL, created_at TIMESTAMPTZ DEFAULT NOW().

3.: hr_practice.employees: emp_id SERIAL PK, emp_name VARCHAR(100) NOT NULL, email UNIQUE NOT NULL, salary NUMERIC CHECK > 0, emp_type DEFAULT 'full_time' CHECK IN ('full_time','part_time','contract','intern'), dept_id INT FK to departments ON DELETE RESTRICT, hire_date DEFAULT CURRENT_DATE.

4.: hr_practice.attendance: Composite PK (emp_id, att_date). FK to employees ON DELETE CASCADE. status DEFAULT 'present' CHECK IN ('present','absent','half_day','leave').

CHALLENGE

Constraints & DML Take-Home Challenge: Build a Constrained HR System (2/2)

Part B: Test Your Constraints

5.: Insert 2 departments and 3 employees.

6.: Test NOT NULL, CHECK, UNIQUE, FK — try 4 invalid inserts.

7.: Test RESTRICT: Try deleting a department with employees.

8.: Test CASCADE: Delete an employee, verify attendance gone.

Part C: DML Practice

9.: UPDATE salary with RETURNING. DELETE by emp_id with RETURNING.

10.: TRUNCATE attendance RESTART IDENTITY. Verify empty.

Push your SQL file to GitHub!

https://github.com/starlordali4444/PostgreSql

REAL WORLD

WHAT YOU ACHIEVED TODAY

You have mastered ALL constraint types: NOT NULL, DEFAULT, UNIQUE, CHECK, PRIMARY KEY, FOREIGN KEY, and CASCADE. You understand referential integrity — why tables CONNECT and how to prevent orphan data. And you can INSERT, UPDATE, DELETE, and TRUNCATE data confidently.

Tomorrow: WHERE, LIKE, ORDER BY, LIMIT — you will finally SEARCH and FILTER data. The first 4 days were building the house. Filtering & Sorting is when you start living in it.

YOUR SQL JOURNEY

- ✓ SQL Intro & PostgreSQL Architecture
- ✓ Installation & RetailMart Onboarding
- ✓ DDL & Data Types — The Column Blueprint
- 4 **Constraints, Keys & DML — Data Integrity** ← HERE
- 5 Filtering — WHERE, ORDER BY, LIMIT
- 8 Aggregates — GROUP BY, HAVING
- 9 Joins — Combining Tables
- 15 earlier sessions Review & Practice

STORY

Coming Tomorrow

You have built tables and filled them with data. But how do you FIND a specific customer? List orders above Rs. 5,000? See the top 10 products by price?

Filtering & Sorting introduces WHERE, LIKE, BETWEEN, IN, IS NULL, ORDER BY, LIMIT — the tools that turn your database from a storage box into a search engine. This is where SQL starts to feel POWERFUL.